

Industry 4.0 Internship



Today's agenda

- **What is MQTT?**
- **Why we use MQTT?**
- **History of MQTT**
- **Publish-subscribe Model**

INTRODUCTION TO MQTT



WHAT IS MQTT?

- It stands for **Message Queuing Telemetry Transport** protocol .
- MQTT is a **Client Server publish/subscribe** messaging transport protocol.
- It is light weight, open, simple, and designed so as to be easy to implement.
- These characteristics make it ideal for use in many situations, including **constrained environments** such as for communication in **Machine to Machine (M2M)** and Internet of Things (IoT) contexts where a small code footprint is required and/or network bandwidth is at a premium.

WHY MQTT?

- In addition to being used as an underlying communications protocol for IoT and Industrial IoT architectures, **MQTT** is used in smart home automation systems alongside cloud platforms such as **Microsoft Azure**, **AWS** and **IBM Watson**.
- **Facebook** also uses MQTT as a communication protocol for its Messenger and Instagram platforms.
- The Industrial Internet of Things (IIoT) can be loosely defined as a system of sensors and other devices interacting with industrial and manufacturing systems all in an effort to enhance business operations. Industries like manufacturing, mining, oil & gas and agribusiness, to name just a few, deploy massive numbers of sensors. These sensors in turn send critical telemetry data to analytics engines, where the data is analyzed for trends and/or anomalies, enabling organizations to better understand and improve their operations.

WHY MQTT?

- In environments using Low Power Wide Area Network (LPWAN) solutions, sensor data is sent over wireless radio transmissions where it is received by one or more central base stations. This data, small as individual packets but massive when aggregated together, is then sent to analytics and visualization tools whether in the cloud or on-premises. That's where MQTT comes in. Residing on top of the TCP/IP network stack, MQTT is a lightweight publish/subscribe messaging protocol designed for low-bandwidth, high latency, unreliable networks. MQTT's features make it an excellent option for sending high volumes of sensor messages to analytics platforms and cloud solutions.

WHY MQTT?

- It is a very **light weight** and **binary protocol**, and due to its **minimal packet overhead(fixed-length header of 2 bytes)**, MQTT excels when transferring data over the wire in comparison to protocols like HTTP.
- Another important aspect of the protocol is that **MQTT** is extremely easy to implement on the **client side**.
- Simple implementation
- Quality of Service data delivery
- Lightweight and bandwidth efficient
- Data agnostic
- Continuous session awareness

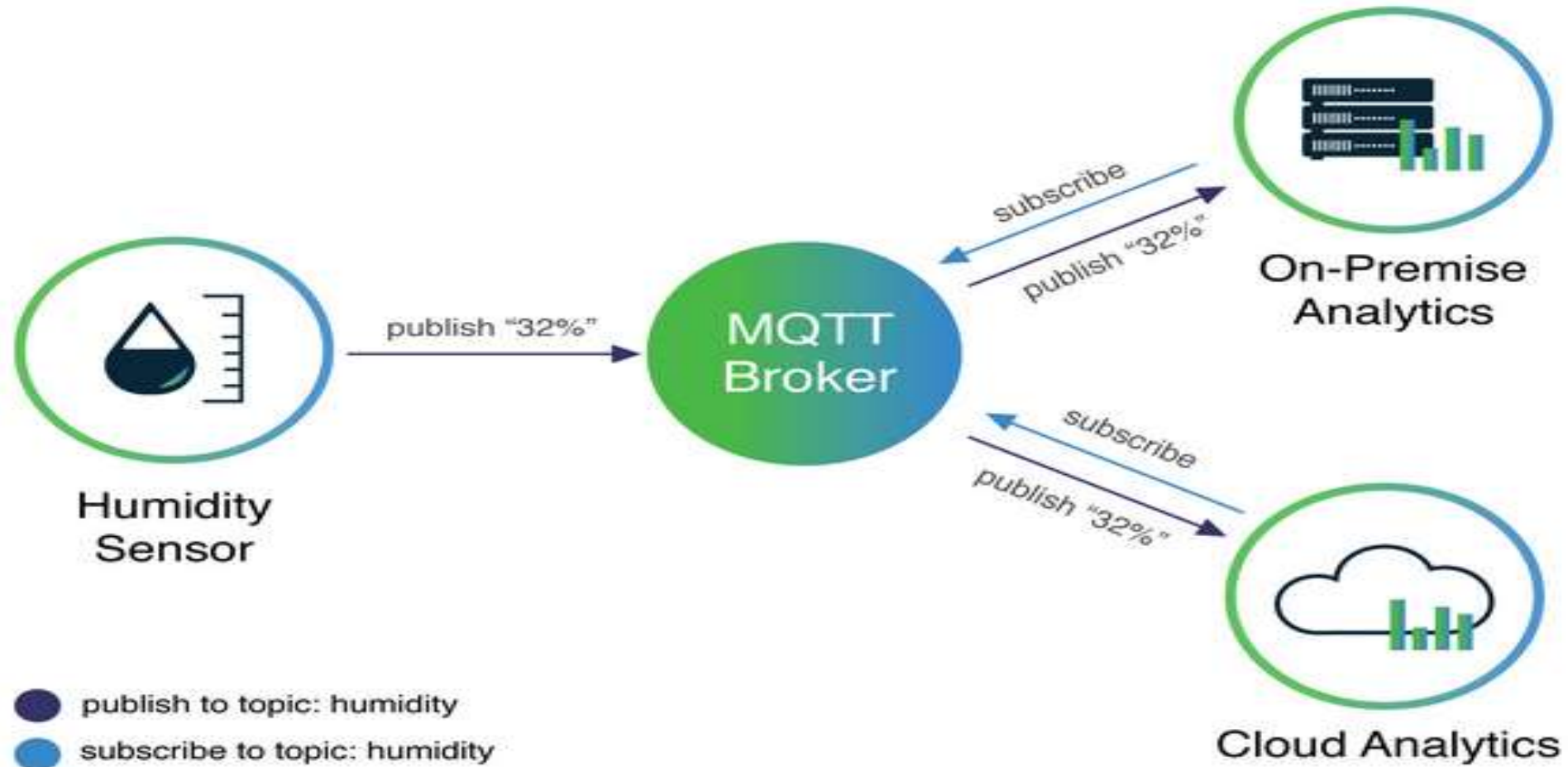
History of MQTT

- MQTT was invented in **1999** by engineers **Andy Stanford-Clark** and **Arlen Nipper**, as a method of allowing pipelines in the oil and gas industry to communicate with Supervisory Control and Data Acquisition (SCADA) systems.
- At the time, these systems used disparate, proprietary protocols, and, as such, were not able to communicate with each other. Adding MQTT capabilities helped overcome inter-communication problems. In addition to interoperability, the original goals for the protocol were that it should be lightweight, bandwidth efficient, data agnostic and simple to implement while offering quality-of-service data delivery.

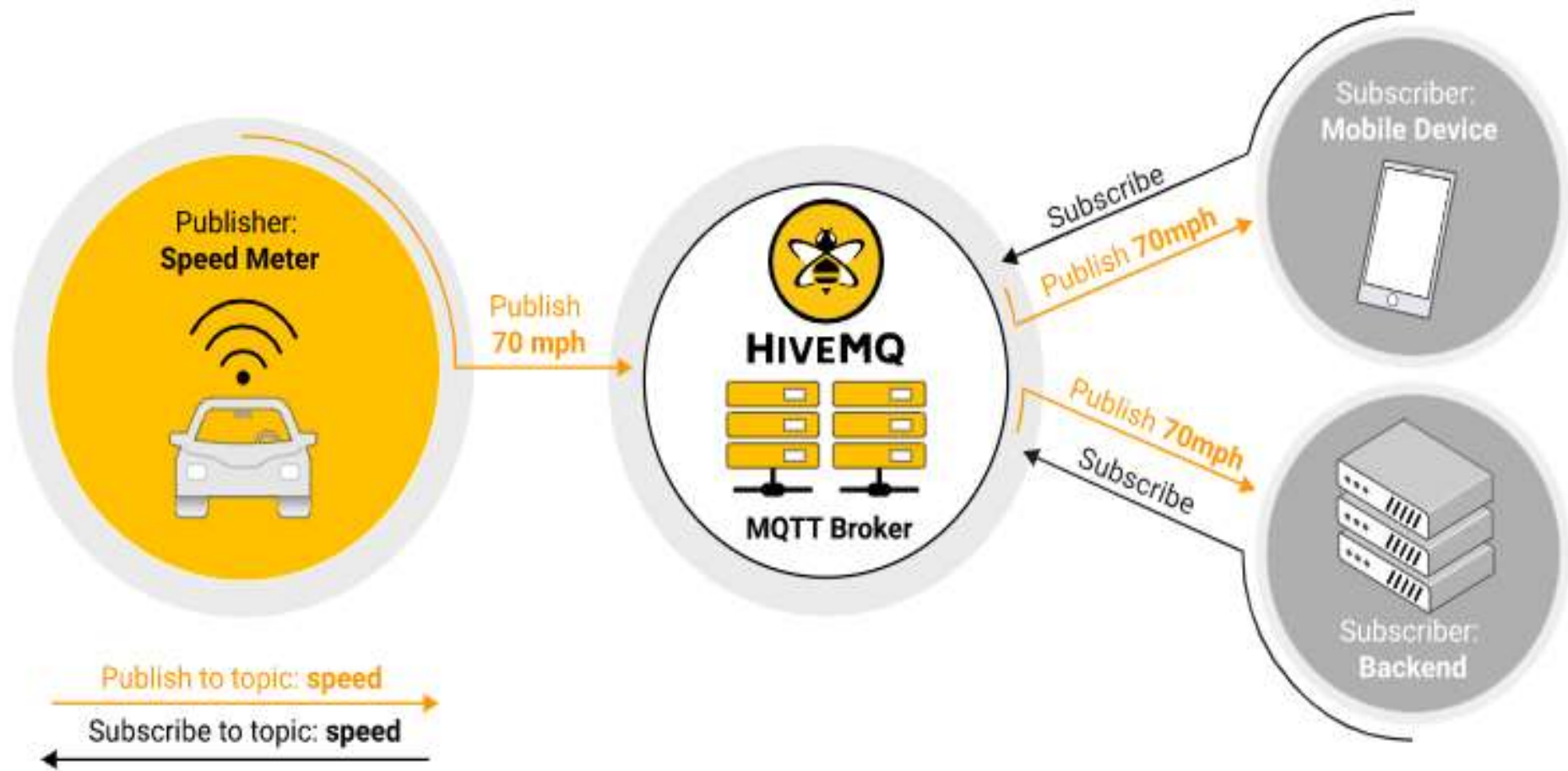
PUBLISH-SUBSCRIBE MODEL

- The **publish-subscribe pattern** (also known as **pub-sub**) provides an alternative to traditional client-server architecture.
- In the client-server model, **a client** communicates directly with an **endpoint** whereas **the pub-sub model decouples** the client that sends a message (**the publisher**) from the client or clients that receive the messages (**the subscribers**).
- The publishers and subscribers never contact each other directly. In fact, they are not even aware that the other exists.
- The connection between them is handled by a third component (**the broker**).
- The job of the **broker** is to filter all incoming messages and distribute them correctly to **subscribers**.

PUBLISH-SUBSCRIBE MODEL



PUBLISH SUBSCRIBE ILLUSTRATION



FEATURES OF PUB-SUB MODEL

- **Decoupling**

The pub-sub model removes direct communication between the publisher of the message and the recipient/subscriber.

- **Scalability**

Operations on the broker can be highly parallelized and messages can be processed in an event-driven way. Message caching and intelligent routing of messages are often a decisive factors for improving scalability.

- **Message filtering**

Filtering is based on the subject or topic that is part of each message.

Also the broker filters the message based on a specific content filter-language. When object-oriented languages are used, filtering is also based on the type/class of a message (event) is a common practice

BASIC TERMINOLOGY BEHIND MQTT

- **Publish/subscribe decouples** the client that sends a message (the publisher) from the client or clients that receive the messages (the subscribers).
- To publish or receive messages, publishers and subscribers only need to know the **hostname/ip** and **port of the broker**.
- MQTT uses the **topic (subject)** of the **message** to determine which message goes to which **client (subscriber)**.
- Every message contains a **topic (subject)** that the broker can use to determine whether a subscribing client gets the message or not.

ESSENTIAL ELEMENTS FOR MQTT

MQTT client

- An MQTT client is any device (from **a micro controller up to a full-fledged server**) that runs an MQTT library and connects to an **MQTT broker** over a network.
- Both **publishers** and **subscribers** are **mqtt clients**.
- The publisher and subscriber labels refer to whether the client is currently publishing messages or subscribing to messages .
- Basically, any device that speaks mqtt over a tcp/ip stack can be called an mqtt client.



ESSENTIAL ELEMENTS FOR MQTT

MQTT Broker

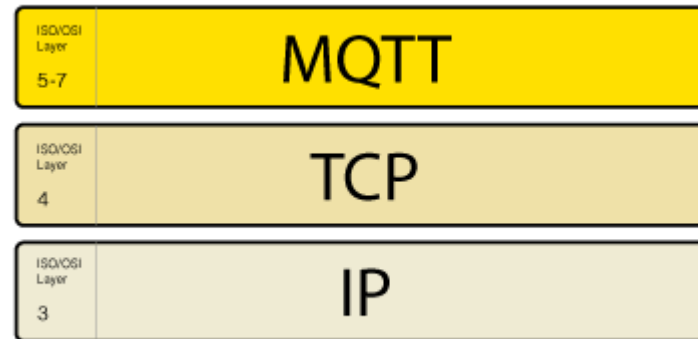
- The broker is at the heart of any publish/subscribe protocol.
- The broker is responsible for receiving all messages, filtering the messages, determining who is subscribed to each message, and sending the message to these subscribed clients.
- A broker can handle up to thousands of concurrently connected MQTT clients.
- The broker also holds the session data of all clients that have persistent sessions, including subscriptions and missed messages.
- Responsible for authentication and authorization of clients.



ESSENTIAL ELEMENTS FOR MQTT

MQTT Connection

- The MQTT protocol is based on TCP/IP. Both the client and the broker need to have a TCP/IP stack.

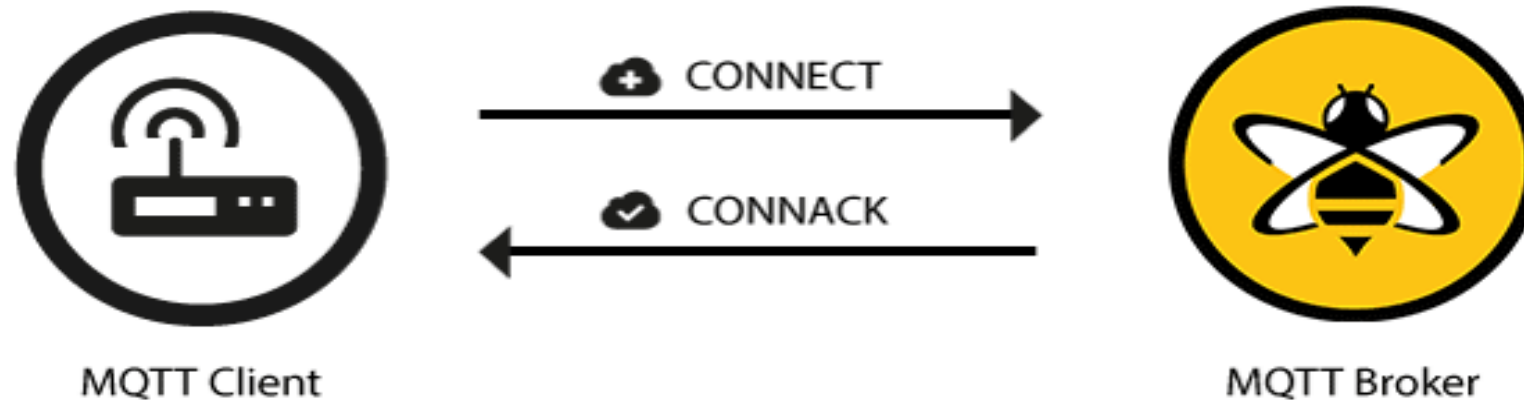


- The MQTT connection is always between one client and the broker. Clients never connect to each other directly.

ESSENTIAL ELEMENTS FOR MQTT

MQTT Connection

- To initiate a connection, the client sends a **CONNECT** message to the broker. The broker responds with a **CONNACK** message and a status code. Once the connection is established, the broker keeps it open until the client sends a disconnect command or the connection breaks.



CONNECT PACKET FORMAT

MQTT-Packet:

CONNECT



contains:

clientId

cleanSession

username (optional)

password (optional)

lastWillTopic (optional)

lastWillQos (optional)

lastWillMessage (optional)

lastWillRetain (optional)

keepAlive

Example

"client-1"

true

"hans"

"letmein"

"/hans/will"

2

"unexpected exit"

false

60

CONNACK PACKET FORMAT

MQTT-Packet:

CONNACK



contains:

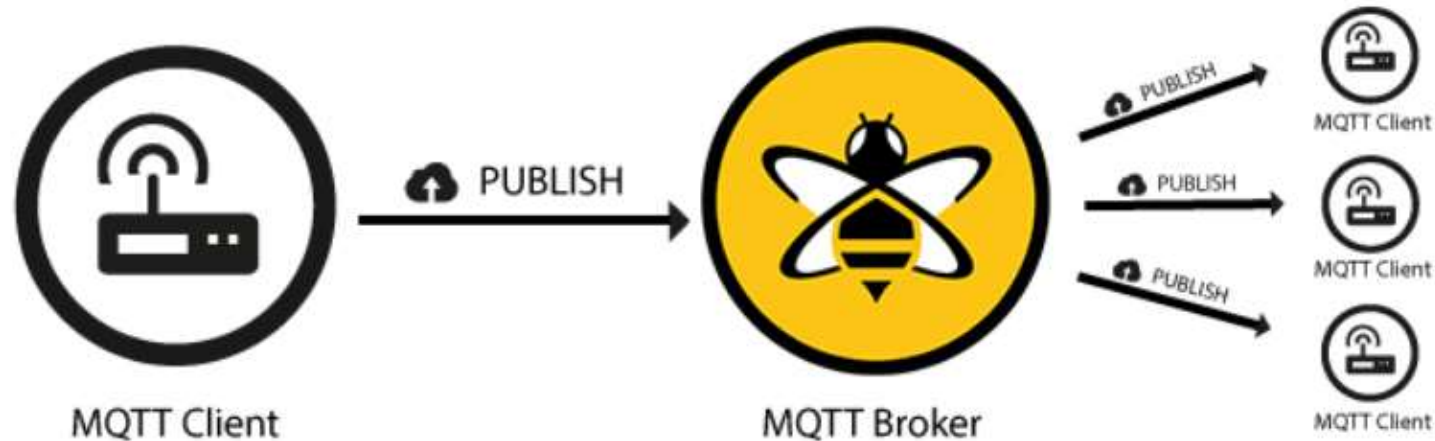
`sessionPresent`
`returnCode`

Example
`true`
`0`

PUBLISH, SUBSCRIBE & UNSUBSCRIBE

PUBLISH

An MQTT client can publish messages as soon as it connects to a broker. MQTT utilizes topic-based filtering of the messages on the broker. **Each message must contain a topic that the broker can use to forward the message to interested clients. Typically, each message has a payload which contains the data to transmit in byte format.** MQTT is data-agnostic. The use case of the client determines how the payload is structured. The sending client (publisher) decides whether it wants to send binary data, text data, or even full-fledged XML or JSON.



PUBLISH PACKET FORMAT

MQTT-Packet:

PUBLISH



contains:

packetId (always 0 for qos 0)

topicName

qos

retainFlag

payload

dupFlag

Example

4314

"topic/1"

1

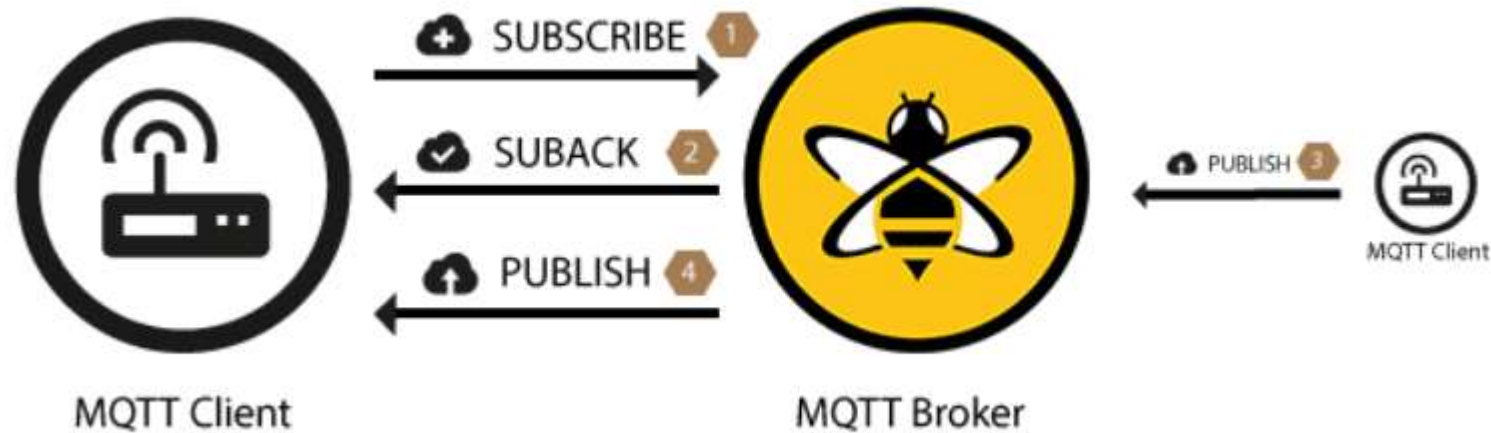
false

"temperature:32.5"

false

SUBSCRIBE

Publishing a message doesn't make sense if no one ever receives it. In other words, if there are no clients to subscribe to the topics of the messages. To receive messages on topics of interest, the client sends a SUBSCRIBE message to the MQTT broker. This subscribe message is very simple, it contains a unique packet identifier and a list of subscriptions.



Mqtt subscribe attributes

Packet identifier the packet identifier uniquely identifies a message as it flows between the client and broker. The client library and/or the broker is responsible for setting this internal MQTT identifier.

List of subscriptions a subscribe message can contain multiple subscriptions for a client. Each subscription is made up of a topic and a qos level. The topic in the subscribe message can contain wildcards that make it possible to subscribe to a topic pattern rather than a specific topic. If there are overlapping subscriptions for one client, the broker delivers the message that has the highest qos level for that topic.



SUBSCRIBE PACKET IDENTIFIER



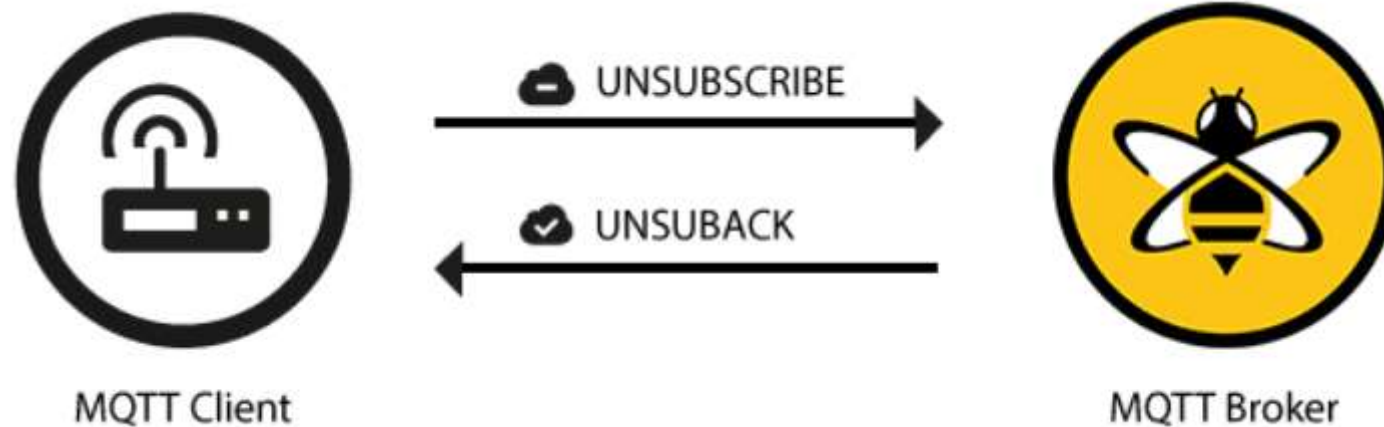
SUBACK

To confirm each subscription, the broker sends a [SUBACK](#) acknowledgement message to the client. This message contains the packet identifier of the original Subscribe message (to clearly identify the message) and a list of return codes.

MQTT-Packet:			
SUBACK			
contains:			Example
packetId			4313
returnCode	1	(one returnCode for each	2
returnCode	2	topic from SUBSCRIBE,	0
...		in the same order)	...

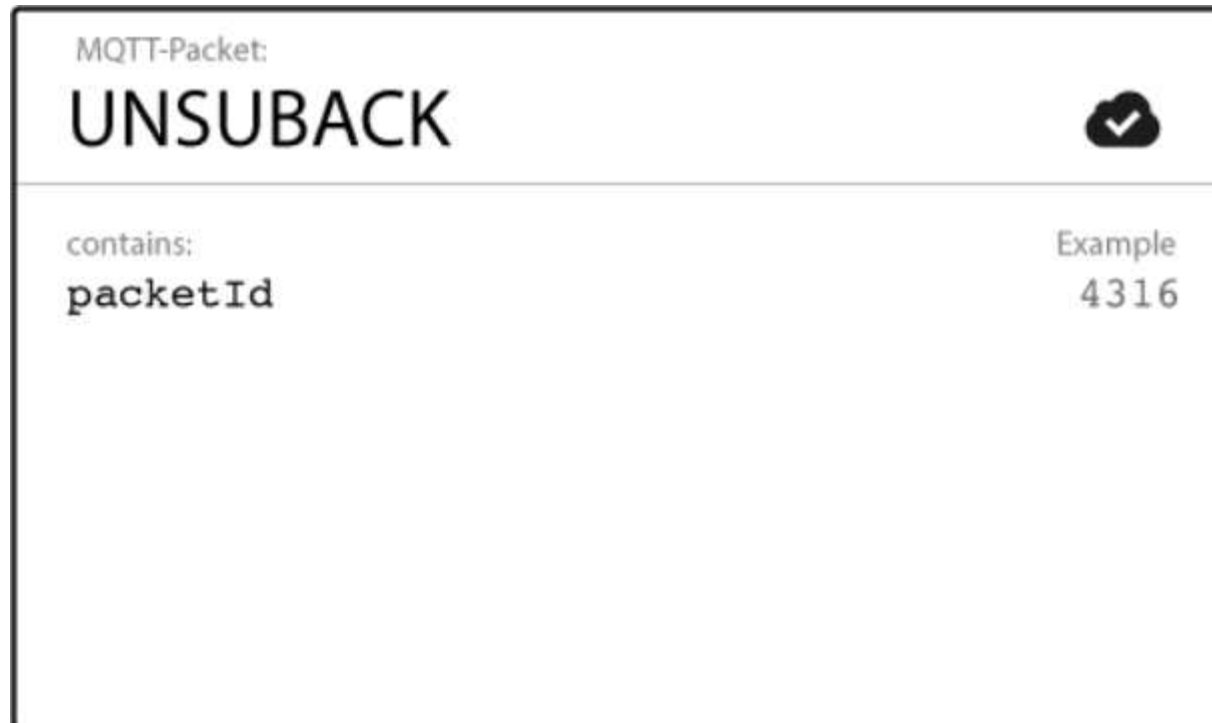
UNSUBSCRIBE

The counterpart of the SUBSCRIBE message is the UNSUBSCRIBE message. This message deletes existing subscriptions of a client on the broker. The UNSUBSCRIBE message is similar to the SUBSCRIBE message and has a packet identifier and a list of topics.



UNSUBACK

To confirm the unsubscribe, the broker sends an UNSUBACK acknowledgement message to the client. This message contains only the packet identifier of the original UNSUBSCRIBE message (to clearly identify the message).



TOPICS

In MQTT, the word topic refers to an UTF-8 string that the broker uses to filter messages for each connected client. The topic consists of one or more topic levels. Each topic level is separated by a forward slash (topic level separator).



In comparison to a message queue, MQTT topics are very lightweight. The client does not need to create the desired topic before they publish or subscribe to it. The broker accepts each valid topic without any prior initialization.

Here are some examples of topics:

myhome/groundfloor/livingroom/temperature
USA/California/San Francisco/Silicon Valley
5ff4a2ce-e485-40f4-826c-b1a5d81be9b6/status
Germany/Bavaria/car/2382340923453/latitude

Note that each topic must contain at least 1 character and that the topic string permits empty spaces. Topics are case-sensitive. For example, *myhome/temperature* and *MyHome/Temperature* are two different topics. Additionally, the forward slash alone is a valid topic.



WILDCARDS

When a client subscribes to a topic, it can subscribe to the exact topic of a published message or it can use wildcards to subscribe to multiple topics simultaneously. A wildcard can only be used to subscribe to topics, not to publish a message. There are two different kinds of wildcards: *single-level* and *multi-level*.

Single level: +

As the name suggests, a single-level wildcard replaces one topic level. The plus symbol represents a single-level wildcard in a topic.

single-level
wildcard
↓
myhome / groundfloor / + / temperature
|
only one level

Any topic matches a topic with single-level wildcard if it contains an arbitrary string instead of the wildcard. For example a subscription to *myhome/groundfloor/+/temperature* can produce the following results:

- ✓ myhome / groundfloor / livingroom / temperature
- ✓ myhome / groundfloor / kitchen / temperature
- ✗ myhome / groundfloor / kitchen / **brightness**
- ✗ myhome / **firstfloor** / kitchen / temperature
- ✗ myhome / groundfloor / kitchen / **fridge** / temperature

Multi level:

The multi-level wildcard covers many topic levels. The hash symbol represents the multi-level wild card in the topic. For the broker to determine which topics match, the multi-level wildcard must be placed as the last character in the topic and preceded by a forward slash.

multi-level
wildcard
↓
myhome / groundfloor / #

only at the end
multiple topic levels



- ✓ myhome / groundfloor / livingroom / temperature
- ✓ myhome / groundfloor / kitchen / temperature
- ✓ myhome / groundfloor / kitchen / brightness
- ✗ myhome / **firstfloor** / kitchen / temperature

When a client subscribes to a topic with a multi-level wildcard, it receives all messages of a topic that begins with the pattern before the wildcard character, no matter how long or deep the topic is. If you specify only the multi-level wildcard as a topic (#), you receive all messages that are sent to the MQTT broker. If you expect high throughput, subscription with a multi-level wildcard alone is an anti-pattern

BEST PRACTICES

- **Never use a leading forward slash**
- **Never use spaces in a topic**
- **Keep the topic short and concise**
- **Use only ASCII characters, avoid non printable characters**
- **Embed a unique identifier or the client id into the topic**
- **Don't subscribe to #**
- **Don't forget extensibility**
- **Use specific topics, not general ones**

